

Computer Engineering 2DX3

Final Project Report

Instructor: Dr. Doyle/Dr. Haddara/Dr. Athar
April 8, 2025

Kishoban Ravendran – L01 – Group 12– ravendk - 400506232

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by **[Kishoban Ravendran, ravendk, 400506232]**

Device Overview Part A: Features

Texas Instrument MSP-EXP432E401Y:

- Used to communicate with the TOF sensor (via I2C) and with the computer (via UART) for the purposes of data visualization
- 32-bit Cortex M4F CPU
- 256 KB SRAM, 1024 KB flash memory
- 120 MHZ bus speed (default), 20 MHZ bus speed (custom for this project)
- GPIO pins
- On-Board LEDS

VL53L1ZX Time of Flight Sensor:

- Operating frequency of 50 HZ
- Max range of 4m
- 2.6-3.5 V operating voltage

28BYJ-48 Stepper Motor:

- 4 phases per step
- 512 steps for 360-degree rotation
- Driver board contains 4 onboard LEDS indicating current phase
- 5-12 V supply voltage

Two Onboard Push buttons/LEDS:

- Configured as active low inputs detected as interrupts by the system
- Push button 1 (PJ0) toggles data acquisition
- Push button 2 (PJ1) toggles motor status and data acquisition state
- PN0 (LED D3) flashing indicates measurement data being transmitted from TOF sensor to microcontroller via I2C
- PN1 (LED D4) flashing indicates measurement data being transmitted from microcontroller to computer
- PF4 (LED D1) periodically blinks throughout scanning process for the sake of bus speed verification
- LED D3 and LED D4 flash every time the motor rotates 45 degrees and collects a measurement

Serial Communication:

- I2C protocol used for communication between TOF sensor and board
- UART protocol used for communication between board and computer (115200 BPS baud rate)

3D Visualization:

- MATLAB facilitates UART communication between microcontroller and computer
- MATLAB code extracts xyz coordinates, writes it into a new file and plots a 3d reconstruction

Device Overview Part B: General Description

The embedded system provides a low-cost, hands-on solution for scanning and visualizing indoor spaces in 3D, embodying the cycle of observation, reason and act. Using a VL531X time-of-flight (ToF) sensor mounted to a stepper motor, the device captures distance data at 8 45-degree intervals across a 360-degree sweep. The sensor communicates with the microcontroller using the I2C protocol.

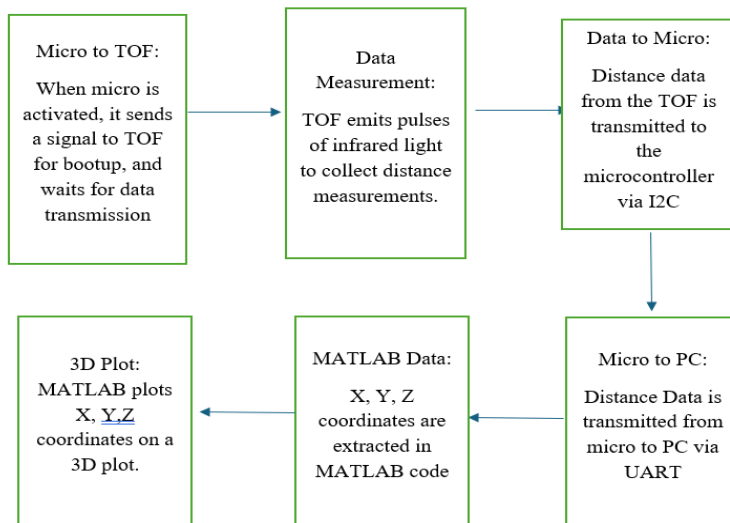
The system is activated by two onboard push buttons connected to Port J on the microcontroller. These push buttons have been configured as interrupts that toggle the state of the overall system. Pressing button 2 (PJ0) enables or disables data acquisition, while button 1 (PJ1) starts or stops the motor and data acquisition process. When data capture is enabled, LED D3 and D4 light up respectively to indicate that the system is collecting and transmitting data. LED D3 indicates that the ToF sensor is transmitting data to the microcontroller via I2C while LED D4 indicates that the microcontroller is transmitting data to the PC via UART. An additional status LED, LED D1, blinks periodically throughout the scanning process to validate the 20 MHZ bus speed configuration. After each complete rotation, the motor reverses direction to prevent cable twisting during continuous use.

Each distance reading is passed from the sensor to the microcontroller via I2C, then sent over UART to a connected PC. A custom MATLAB script running on the PC receives data from the serial port (COM5), parses the incoming data, and extracts the (X, Y, Z) coordinates. Within the Keil code, the x-axis offset is incremented after each full scan to simulate linear motion. The y-axis and z-coordinates are calculated within the code through trigonometric calculations using the distance measurement data from the ToF sensor.

The device runs on a student-assigned bus speed of 20 MHz and uses a polling + interrupt-based design for handling sensor reads, motor control, and user input.

Once the data is collected, it is stored in a .xyz file and visualized in MATLAB using a 3D plot. This allows the user to clearly see the shape and layout of the scanned area. The system is designed to be lightweight and portable for mapping hallways, corners, or other indoor locations, and can be used as a foundational tool for applications like robotics or navigation systems.

Device Overview Part C: Block Diagram



Device Characteristics

Microcontroller	Stepper Driver	Time of Flight (ToF)
Clock Speed: 20 MHZ	In1: PH0	Vin: 3.3V
Measurement and Status LEDS (PN0, PN1, PF4)_	In2: PH1	GND
Onboard Push Buttons (PJ0 and PJ1)	In3: PH2	SDA: PB3
Serial Port: COM5	In4: PH3	SCL: PB2
Baud Rate: 115200 BPS	V+: 3.3V, V- : GND	

Detailed Description Part A – Distance Measurement:

The system uses a VL53L1X Time-of-Flight sensor to measure distance based on the time it takes for a beam of infrared light to reflect off a surface and return to the sensor. The sensor automatically calculates the measured distance using the formula:

$$\text{Measured Distance} = \frac{\text{Photon Travel Time}}{2} * \text{Speed of Light}$$

The VL531X is interfaced with the MSP-EXP432E401Y microcontroller over I2C (using I2C0 on Port B). Once the sensor completes its startup and is properly initialized, it begins continuously ranging. Every time the microcontroller receives confirmation that the measured data is ready (this is achieved through a polling method in the Keil code), the stepper motor movement is paused, and the microcontroller retrieves the distance in millimeters.

Distance measurements are triggered at 45-degree intervals during a full 360-degree sweep. The stepper motor, which is connected to the micro via Port H (PH0-PH3), is rotated by 64 steps between each reading. This results in eight distance measurements being collected per full rotation. To keep the wiring untangled, the system rotates back 360 degrees counterclockwise after every full sweep clockwise.

User input is handled using external buttons connected to port J:

- PJ0 toggles the data acquisition mode
- PJ1 starts/stops the full motor + scan process

The system uses interrupt-based detection for both buttons. While the scan is active and data acquisition is enabled, each sensor reading is captured, and the associated data point is transmitted over UART.

Real-time status is showing using onboard LEDs:

- PN0 (LED D3): Blinks when distance is being transmitted from ToF to micro (I2C)
- PN1 (LED D4): Blink when distance is being transmitted from micro to PC (UART)
- PF4 (LED D1): Flashes to verify the correct bus speed and indicate overall speed of system

Detailed Description Part B – Visualization:

Once a measurement is taken, the data is formatted as:

$$Index, Angle(Deg), X(mm), Y(mm), Z(mm)$$

The line is sent via UART to the PC. The MATLAB script waits to receive the data over the serial port (COM5), parses each line, and extracts the X,Y, and Z values.

Coordinate conversion is handled on the Keil Code:

- Angle (deg) is calculated based on the scan index
- Angle (rad) is used to compute polar-to-Cartesian conversion
- X is a fixed linear offset that increments by 100mm with each new scan
- Y and Z are computed using:

$$Y = Distance * \cos(\theta) \quad Z = Distance * \sin(\theta)$$

The MATLAB script stores each coordinate in an array and writes it to a .xyz file. After the scan is complete, the script automatically generates a 3D scatter plot using a function called plot3(), connecting points in sequence to visualize a 3D profile of the scanned area.

Each full rotation produces 8 distance measurements. As the X-offset increments after each rotation, the device performs a linear “slice” scan at fixed intervals, building a 3D point cloud of the environment over time.

The scan output is appropriate for visualizing hallway layouts, corners, or other indoor structures. The resolution of the 3D visualization is determined by the number of steps on the motor and the number of scans performed.

Application Note:

Embedded scanning systems that use time-of-flight (ToF) technology can accurately measure distances and generate 3D models of indoor environments. These systems are lightweight, low-cost, and are capable of real-time spatial data collection. This in turn makes them suitable for a variety of engineering and industrial applications.

One application of ToF-based scanning systems is in indoor navigation and robotics. Mobile robots rely on spatial awareness to avoid collisions and plan efficient paths through unfamiliar environments (Observe, Reason, Act). By integrating a ToF sensor with a microcontroller and precise motor control, autonomous systems can constantly update their observational understanding of the surrounding environment, enabling them to operate safely in tight corridors or crowded rooms.

This technology is also valuable in facility management and maintenance. The ability to create 3D maps of building interiors can assist in planning renovations, identifying spatial inefficiencies, or validating architectural layout. These types of systems are helpful in older facilities that lack up-to-date blueprints or CAD models.

Whether used for robotic perception or spatial documentation, ToF-based scanning platforms offer an accessible solution for capturing the 3D shape and layout of complex environments.

Instructions:

Power MSP-EXP432E401Y microcontroller:

- Connect the board to a PC via USB using specific port
- Ensure the board is powered and the system clock is configured (20 MHZ)

Check Component Connections:

- VL53L1X Time-of-Flight
 - $V_{in} = 3.3V$
 - GND = GND
 - SDA = PB3
 - SCL = PB2
- Stepper Motor
 - IN1 = PH0
 - IN2 = PH1
 - IN3 = PH2
 - IN4 = PH3

PC Code:

- Build, translate, load Keil script
- Open the corresponding MATLAB code
- Make sure the serial Port variable is set to the correct COM port for the micro board (COM5)
- Run the MATLAB script before starting the scan
- The script will output live data to the console and save a file named `tof_scan_data.xyz`

Operating the System:

- Enable data acquisition and motor control through pressing button 1 (PJ1)
- The motor will begin a 360-degree clockwise scan, pausing every 45 degrees to take a measurement
- After 8 readings, the motor will reverse direction (counterclockwise) and return to home position to prevent wire tangling
- LED D3 (PN0) blinks with each I2C read and LED D4 (PN1) blinks with each UART read
- LED D1 (PF4) flashes continuously for system bus speed verification
- Press button 1 to stop scan
- (Optional) Press button 2 (PJ0) to toggle off data acquisition mode while still keeping motor running

Review and Save Output:

- Data appears live in MATLAB window
- Saved .xyz file will contain rows formatted as: X(mm), Y(mm), Z(mm) representing a point in 3D space
- After scanning, MATLAB will automatically plot the result using plot3() in 3D space
- Plot should visually resemble the scanned structure

Expected Output:

The expected output can be visualized by comparing my assigned scanning location (location C), to the 3D scanned representation. Below is a picture of the real life hallway, and beside it is the reproduced 3D scan.



Figure 1: Real-life Hallway

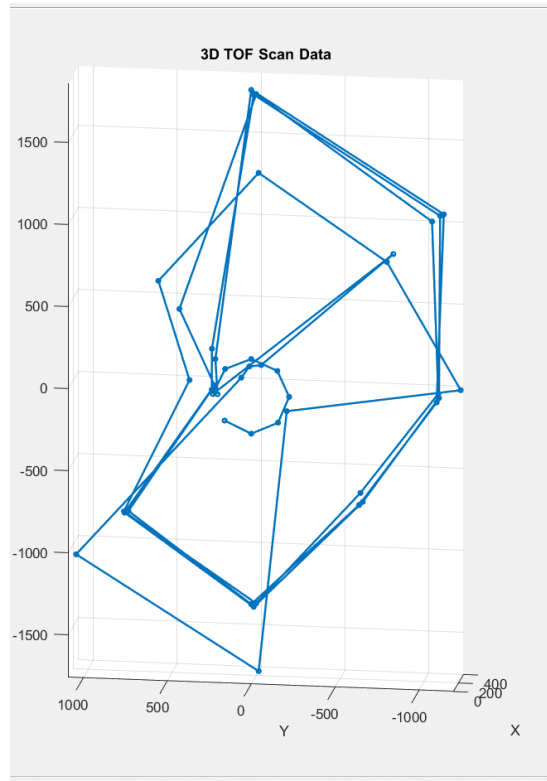


Figure 2: Reproduced 3D Scan

As seen in the finalized scan, the scanning system can somewhat capture the details of the scanned location. Figure 2 showcases that, to some extent, the general shape of the hallway was captured. This hallway is relatively narrower, which the 3D scan represents through being somewhat compact as well. The sides of the hallway contain mostly doors, which is represented in the 3D scan as there are spikes in the spatial measurements on the side. At one point, the sensor mount fell off the motor, causing the scanning system to exclusively detect a perfect circle of rotation at one point. There were also certain points where the motor stopped for a brief period of time during the scanning process due to loose wire connections.

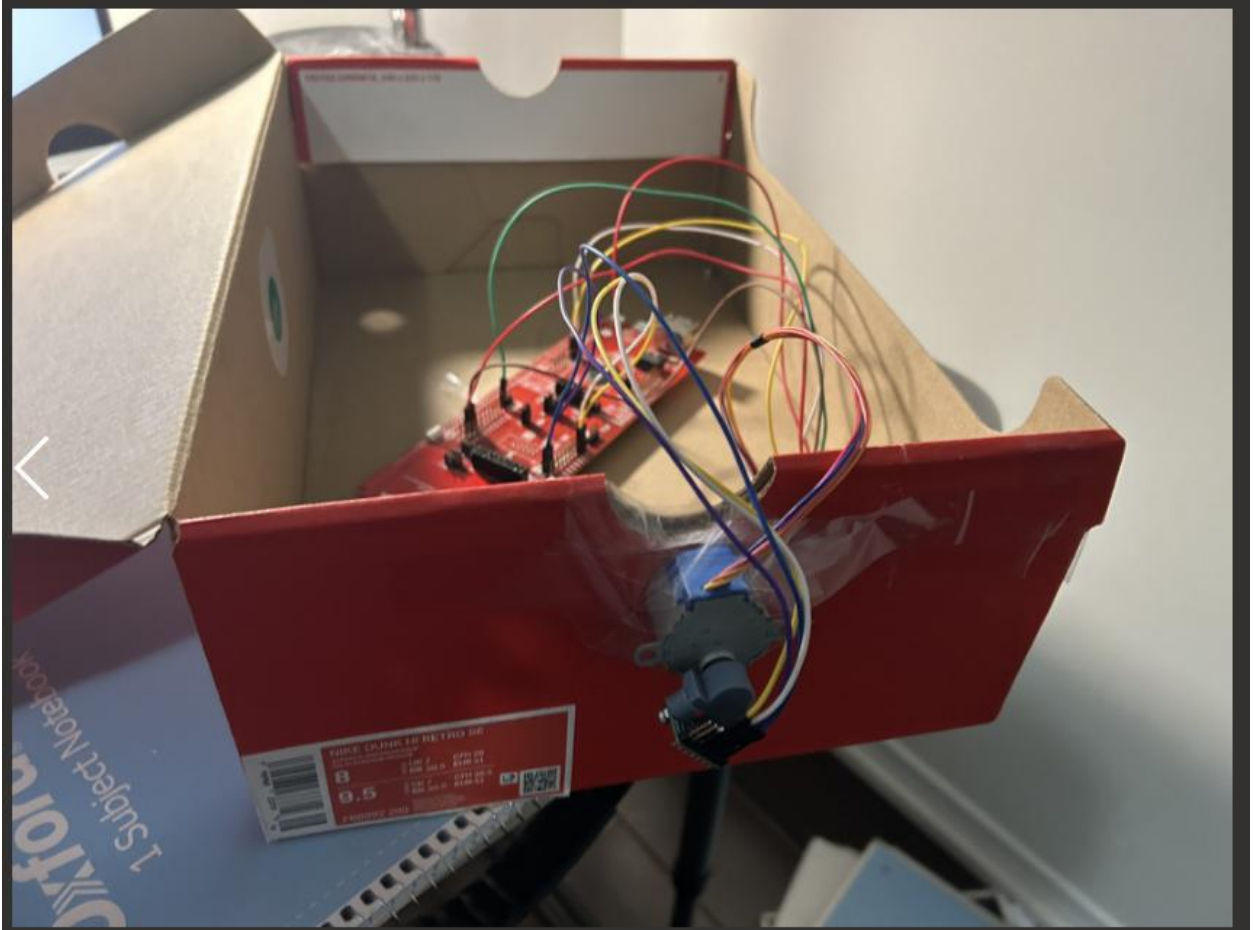


Figure 3: Physical Scanning System

Limitations:

In code implementation, operations involving trigonometric functions such as $\sin()$ and $\cos()$ are used; however, they are more computationally expensive than basic arithmetic operations and can introduce small rounding errors. In this application, these trigonometric functions are used to convert polar distance measurements into Cartesian coordinates (Y and Z axes). Since accuracy beyond 2 decimal places is not important for this situation, this performance trade-off is acceptable.

Max Quantization is equal to resolution:

$$(VFs)/(2^{(\text{number of bits})})$$

$$\text{ToF res: } \frac{5}{2^8} = 19.53 \text{ mV}$$

$$MCU\ res = \frac{5}{2^{12}} = 1.22\ mV$$

$$Stepper\ Motor\ res = \frac{5}{2^{16}} = 76.30\ \mu V$$

The microcontroller supports standard UART baud rates up to 3Mbps, however, in this implementation, a baud rate of 115200 bps was used for compatibility and reliability. To verify, the UART output was visualized in real time in Real Term, which was specifically configured to have a baud rate of 115200 bps. When higher speeds were configured on Real Term, transmission errors were detected.

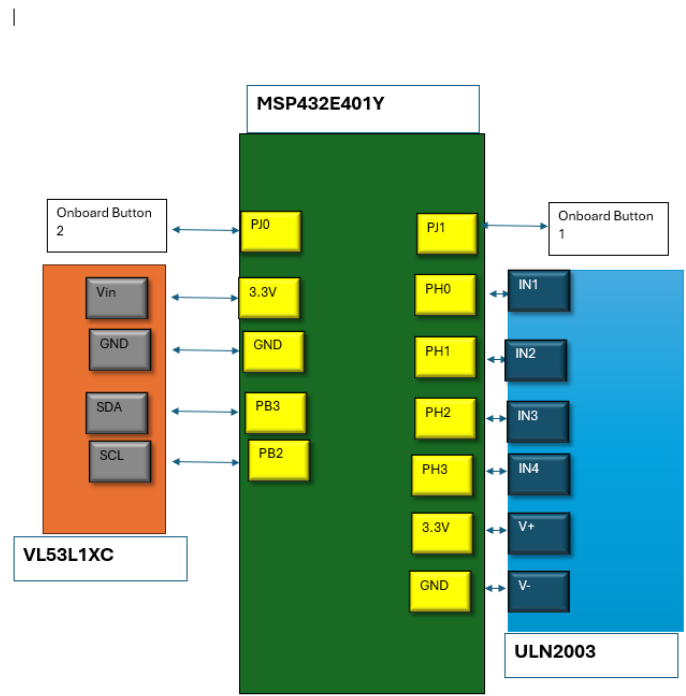
The VL53L1X communicates over the microcontroller over the I2C bus with a speed of I2C speed of 400 kHz. The specific I2C pins that are connected to the micro are SCL(PB2), SDA(PB3). This sensor is addressed using the default I2C address 0x29.

The speed of the microcontroller, more specifically the speed of the stepper motor, has the greatest impact on the performance of the scanning system. These directly affect the speed at which specific data measurements are collected.

For a bus speed of 20 MHz, I had to edit PLL.h by changing the divisor PSYDIV.

$$Bus\ speed = \frac{400\ Mhz}{PSYDIV + 1}; 20\ Mhz = \frac{400\ Mhz}{PSYDIV + 1}; PSYDIV = 23$$

Circuit Schematic:



Programming Logic Flowchart:

